**SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)**

**SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE**

*Department of Artificial Intelligence and Machine Learning*

# A PROJECT BASED LEARNING (PBL) REPORT

ON

# AI Edu Assessment: A Retrieval-Augmented Platform for Document Question Answering and Automatic Quiz Generation

*Submitted in partial fulfilment of the Mini Project / PBL requirement*

*for the degree of*

**BACHELOR OF TECHNOLOGY**

in Artificial Intelligence and Machine Learning

**Academic Year 2026–27**

## GROUP DETAILS

| Particular | Details |
|---|---|
| Group Number | _____ (to be filled by the group) |
| Project Title | AI Edu Assessment |
| Domain | Natural Language Processing / Educational Technology |
| Type of Project | Application-oriented mini project (PBL) |
| Internal Guide | _____ (to be filled) |
| Semester / Year | _____ |

*Table 1. Group and project identification*

Names, PRNs and roles of all members (any one member may submit this report on behalf of the group):

| Sr. No. | Name of the Student | PRN | Proposed Role |
|---|---|---|---|
| 1 | Sukhada Dhingra | _____ | Literature survey, report, product flow |
| 2 | Dipti Kothari | _____ | Backend API, readability and glossary |
| 3 | Khurinji Malar | _____ | Mobile application (React Native / Expo) |
| 4 | Akhil | _____ | RAG pipeline, embeddings, quiz module, testing |

*Table 2. Group members with PRN (fill PRNs before submission)*

*Note: Permanent Registration Numbers (PRN) and the official Group Number were not available at the time of drafting. Each member must enter her/his PRN in Table 2 and the Group*

*Number in Table 1 before the report is uploaded. Only one copy of this report is to be submitted for the whole group.*

**Month of Submission: September 2026**

## 1. ABSTRACT

This report presents the problem definition, motivation, objectives, literature study, proposed method and work plan of our PBL mini project titled AI Edu Assessment. The project deals with a practical difficulty that students and teachers face while working with PDF notes, research papers and textbooks: it is slow to search for answers inside a long file, and it is even slower to prepare a fair set of practice questions from the same file. Generic chat tools often invent facts that are not present in the source. Our proposed system takes a PDF (by URL or upload), splits the text into overlapping sentence-aware chunks, stores vector embeddings of those chunks in a Pinecone index, and then answers user questions only from the retrieved passages. The same retrieved text is used to generate multiple-choice quizzes. Alongside the retrieval pipeline, the backend also estimates reading difficulty using the Flesch formulae and builds a short glossary of frequent terms. A React Native (Expo) client is planned so that the tool can be used from a phone. The work is grounded in published research on retrieval-augmented generation, dense passage retrieval, sentence embeddings, automatic question generation and readability. This document is the group's single submission for the mini-project proposal stage.

*Keywords: retrieval-augmented generation, dense embeddings, automatic quiz generation, educational assessment, FastAPI, React Native, Pinecone, Gemini.*

## 2. INTRODUCTION

Reading a PDF from beginning to end is not the same as studying it. In a typical week an engineering student may download lecture slides, a research paper from arXiv, a chapter from a textbook and a set of previous-year questions. Finding one definition inside fifty pages, or converting that chapter into a short test, still takes a long time. Search inside a PDF is mostly keyword based. If the wording of the query does not match the wording of the author, the relevant paragraph is missed. Teachers have a related problem: writing a new quiz after every unit is extra work, and questions copied from the internet may not match the exact notes that were taught.

Large language models can produce fluent answers and questions, but they do not automatically know what is written in a particular file that the user has opened. When the model is asked about that file without giving it the right passages, it fills the gaps from its training memory. For assessment this is unsafe. A student may memorise an answer that was never in the notes. A teacher may print a question whose correct option is factually wrong with respect to the prescribed text.

Retrieval-Augmented Generation (RAG) is a way of reducing that gap. The idea, stated simply, is: first find the paragraphs that are close in meaning to the user's question, then ask the language model to write the answer using only those paragraphs. The same idea can be used for quizzes. Instead of asking the model to "make five questions on machine learning", we retrieve several chunks from the ingested PDF and instruct the model to write items that a student could answer from those chunks. Our mini project applies this pattern to educational PDFs and wraps it in a mobile-first interface.

The rest of the report is organised as follows. Section 3 states the problem. Section 4 explains why the group chose this problem. Section 5 lists measurable objectives. Section 6 reviews more than ten research papers that shaped the design. Section 7 describes the proposed methodology, modules, data flow and tools. Section 8 gives the semester timeline. Section 9 records expected outcomes, limitations and a short conclusion. References follow IEEE style.

## 3. PROBLEM STATEMENT

Students and teachers who work with educational PDFs do not have a lightweight, source-grounded tool that can (i) accept a document, (ii) answer natural-language questions using only the content of that document, and (iii) produce a small set of practice questions from the same content. Existing options fall into three weak groups. First, in-file search depends on exact words and does not understand a rephrased doubt. Second, generic chatbot applications may give a confident answer that is not supported by the uploaded notes. Third, commercial quiz makers either need questions to be typed by hand or generate items from a topic name rather than from the actual pages the class has studied.

The problem we take up in this mini project is therefore the following:

*To design and implement a retrieval-augmented educational assessment system that ingests a PDF, indexes its text as dense vectors, answers user questions with answers grounded in retrieved passages, and automatically composes multiple-choice quiz items from the same document, accessible through a mobile application.*

The problem is bounded for a mini project. We will not attempt a full learning-management system, user accounts, or grading of handwritten answers. Image-only pages, scanned books without a text layer, and complex tables are outside the first version. The success of the work will be judged by whether a user can ingest one PDF, ask a question in ordinary English, receive an answer that can be traced to retrieved chunks, and obtain a short quiz that a peer could attempt after reading the same file.

## 4. MOTIVATION

The first reason for choosing this topic is daily academic load. Members of the group regularly download papers and notes as PDFs. Preparing for internals often means highlighting a file and hoping that the highlighted lines are the ones that will be asked. A tool that can be queried in the same language as a classroom doubt, and that refuses to answer from outside the file, would save time without encouraging blind copying from the internet.

The second reason is assessment quality. A quiz is useful only when it matches what was taught. Automatic question generation has a long research history, from template filling to neural sequence models [6][7]. Recent work shows that teachers can steer language models with Bloom's taxonomy to obtain items at different cognitive levels [10]. Those papers also show that ungrounded generation drifts. Grounding the generator in retrieved chunks is a practical way to keep items close to the prescribed material.

The third reason is technical learning that matches our programme. The department expects PBL work to use methods taught in AI and ML rather than a purely cosmetic application. Building this system forces us to handle text extraction, chunking, embedding models, vector search, prompt design, API design and a mobile client. These are skills that appear in internships and in later courses on NLP.

The fourth reason is a known weakness of language models that the literature has already named. Lewis et al. [1] pointed out that parametric models store knowledge inside weights, which makes updating and citing sources difficult. RAG keeps a separate non-parametric memory (the document index) so that new notes can be added without retraining. For a college mini project this is realistic: we cannot fine-tune a large model on every PDF a student uploads, but we can embed the PDF and retrieve from it.

Finally, there is a local need. Many classmates study on phones rather than laptops. A React Native client, talking to a FastAPI service, is a better fit than a desktop-only tool. If the pipeline works on lecture PDFs used in our own courses, the same method can later be shown to faculty as a way to draft unit tests.

## 5. OBJECTIVES

The overall aim is to build a working prototype of AI Edu Assessment within one academic semester and to document the design decisions against published work. Specific objectives are stated below in a form that can be checked at the end of the project.

O1. Document ingestion. Accept a PDF through a URL (and, where feasible, file upload), extract text, and reject empty or unsupported files with a clear error.

O2. Chunking. Split extracted text into overlapping, sentence-aware chunks (target size about 500 words with about 100 words of overlap) so that a concept is not cut in the middle of a sentence.

O3. Indexing. Encode each chunk with a dense embedding model (intfloat/e5-large-v2, 1024 dimensions, query/passage prefixes) and store vectors with metadata in a Pinecone index using cosine similarity.

O4. Grounded question answering. For a user question, retrieve the top-k similar chunks and generate an answer with a Gemini model using a prompt that instructs the model to stay within the given context.

O5. Quiz generation. Produce a configurable number of multiple-choice questions (about 1 to 20) from retrieved document contexts, returned as structured JSON that the mobile client can render.

O6. Study aids. Compute Flesch Reading Ease and Flesch–Kincaid Grade Level for the text, and build a short glossary of frequent terms with definitions.

O7. Mobile access. Provide a React Native (Expo) interface for ingest, question answering and quiz attempt, talking to the FastAPI backend over HTTP.

O8. Evaluation in the mini-project sense. Demonstrate the flow on at least two educational PDFs, record response behaviour (success, empty retrieval, API failure), and write the final report and demo.

Out of scope for this semester: multi-user authentication, persistent personal libraries, OCR for scanned books, extraction of figures and tables as first-class objects, and automated marking of free-text answers.

## 6. LITERATURE REVIEW

A literature review for this project has to cover four connected threads: how a language model can be given external memory, how passages are found quickly, how questions are written for learners, and how text difficulty is measured. We read more than ten papers and a survey. The discussion below is organised by theme rather than as a list of abstracts. A comparison table is given at the end of the section.

## 6.1 Open-domain QA before RAG

Chen, Fisch, Weston and Bordes [8] described DrQA, a pipeline that first retrieved Wikipedia paragraphs with sparse methods (TF-IDF style) and then ran a reader to extract an answer span. The paper made two points that still matter. First, open-domain QA is a retrieve-then-read problem, not a single neural net that "knows" the whole encyclopedia. Second, the quality of the reader is wasted if the retriever brings the wrong paragraph. Our project is smaller than Wikipedia-scale QA, but the same split is used: a retriever (Pinecone + embeddings) and a generator (Gemini).

Sparse retrieval has an obvious limit. If a student asks "How does attention avoid recurrence?" and the PDF says "the Transformer dispenses with recurrence and convolutions entirely," keyword overlap is weak. That observation is the starting point of dense retrieval work discussed next.

## 6.2 Dense retrieval and sentence embeddings

Reimers and Gurevych [3] showed that using BERT in a naive way (averaging token vectors, or taking the CLS token) gives poor sentence similarity. Their Sentence-BERT architecture trains Siamese networks so that cosine similarity between two sentence vectors actually ranks meaning. Almost every later embedding model used in RAG tutorials, including the one we plan to use, sits in this line of work: produce a single vector per passage and compare with cosine or inner product.

Karpukhin et al. [2] asked whether a dual-encoder, trained on question–passage pairs, could replace BM25 for open-domain QA. Dense Passage Retrieval (DPR) encoded questions and passages separately and used inner-product search. On several QA sets the dense retriever beat BM25 by a large margin in top-k accuracy. For our mini project we do not have the resources to train a DPR model on educational PDFs. We therefore take a strong off-the-shelf embedder and keep the dual-encoder idea: one vector for the query, one vector per chunk.

Wang et al. [4] released the E5 family of text embeddings, trained with weakly supervised contrastive learning on large text-pair data. A practical detail in E5 is the use of prefixes: "query:" for questions and "passage:" for documents. If the prefixes are dropped, retrieval quality falls. We adopt intfloat/e5-large-v2 (1024 dimensions) for this reason, and the methodology section treats the prefixes as a required step, not an optional string.

## 6.3 Retrieval-augmented generation

Lewis et al. [1] introduced RAG as a fine-tuning recipe that combines a parametric seq2seq model with a non-parametric dense index (originally Wikipedia). They compared a variant that uses the same retrieved passages for the whole output with a variant that may change passages per token. The result that we take from the paper is not the exact BART setup, but the claim that generation conditioned on retrieved text is more specific and more factual than generation from the model weights alone. That claim is the scientific justification for not sending a whole PDF into a prompt and hoping for the best, and for not asking Gemini about a lecture file without retrieval.

Guu, Lee, Tung, Pasupat and Chang [9] proposed REALM, which pre-trains a language model together with a retriever so that retrieving useful documents is part of the training objective. REALM is heavier than what a semester project can reproduce. We cite it because it shows a second, more expensive path: jointly training retrieval and generation. Our design is the cheaper path used in most applied systems after 2023: a frozen embedder, a vector database, and a frozen instruction-tuned LLM.

Izacard and Grave [12] argued that extractive readers waste the power of generative models. Fusion-in-Decoder (FiD) encodes many retrieved passages independently and fuses them in the decoder. The lesson for us is about context packing. We cannot implement FiD, but we do retrieve several chunks (top-k about 5 for answers, a larger pool for quizzes) and concatenate a bounded amount of text into one prompt. If too many chunks are stuffed in, the model may ignore the middle. If too few are used, the answer is incomplete. The survey by Gao et al. [5] later named this family of design choices (chunk size, overlap, top-k, reranking, prompt format) as the “RAG pipeline” that practitioners actually tune.

Gao et al. [5] is a survey rather than a single algorithm paper. It is useful for a student project because it separates naive RAG (retrieve once, generate once) from more advanced loops such as rewrite-then-retrieve. For a mini project we stay with naive RAG. The survey also lists failure modes: retrieval of similar but wrong passages, hallucination when the context is empty, and loss of structure when PDFs are converted to plain text. Those failure modes appear again in our limitations.

### 6.4 Automatic question generation for education

Kurdi, Leo, Parsia, Sattler and Al-Emari [6] reviewed automatic question generation (AQG) for educational use. They grouped methods into template-based, syntax-based and semantic/neural approaches, and they noted that evaluation is often weak: many papers report BLEU against a reference question rather than asking whether a teacher would accept the item.

They also observed that generating a question is easier than generating a question plus a correct answer plus plausible distractors. That last point is why our quiz module must return options, not only a stem.

Du, Shao and Cardie [7] treated question generation as a sequence-to-sequence task over a reading-comprehension passage. Their model learned to ask questions that a person might ask after reading a paragraph. The paper is from 2017, before instruction-tuned LLMs, but the data assumption is the same as ours: the question should be answerable from a given passage. We replace their trained seq2seq model with a prompted Gemini call, which is a practical choice for a four-person group, not a claim that prompting is scientifically superior to a dedicated QG model.

Elkins, Kochmar, Cheung and Serban [10] studied how teachers can use large language models together with Bloom's taxonomy to create quizzes. The important result for us is pedagogical rather than architectural. Teachers wanted control over cognitive level (remember, understand, apply, and so on) and they still had to edit model output. We do not claim that our generator will replace a teacher. We claim that a first draft of MCQs, tied to a specific PDF, is a reasonable assistant. If time permits in later weeks, the prompt can ask for a mix of recall and understanding items; that is optional, not Objective O5.

### 6.5 Readability as a study signal

Flesch [11] proposed a reading-ease formula based on average sentence length and average syllables per word. The Flesch–Kincaid grade level, which is widely used in education software, is a linear rearrangement of similar counts. These formulae are old and they ignore domain vocabulary: a short sentence full of defined technical terms can still be hard. We still include them because they are cheap, transparent, and familiar to teachers. A student who uploads a paper and sees a high grade-level score gets a rough warning that the text is dense, which is a small but honest extra feature next to Q&A and quizzes.

### 6.6 Comparative summary and gap

Table 3 places the papers against the modules we need. No single paper is a complete educational product. DrQA and DPR stop at retrieval and reading. RAG, REALM and FiD stop at knowledge-intensive NLP, not at quiz UX. AQG papers stop at item writing and rarely store a per-student PDF index. Teacher-facing LLM quiz papers rarely specify a vector database. The gap we try to fill is therefore an integration gap: a small, document-grounded loop from PDF to

answer and PDF to quiz, with a phone client, using published RAG practice rather than inventing a new training method.

| Ref. | Work | Year | What we take from it | What it does not solve for us |
|---|---|---|---|---|
| [1] | Lewis et al., RAG | 2020 | Answers should be generated from retrieved memory, not only from model weights. | No educational UI; original setup uses Wikipedia and BART. |
| [2] | Karpukhin et al., DPR | 2020 | Dense dual-encoder retrieval beats keyword search for questions. | Needs specialised training data; we use a frozen embedder. |
| [3] | Reimers & Gurevych, SBERT | 2019 | Sentence vectors must be trained for cosine similarity. | Not a full QA or quiz system. |
| [4] | Wang et al., E5 | 2022 | Prefix-based embeddings (query / passage) and 1024-d E5-large. | Does not generate answers or questions. |
| [5] | Gao et al., RAG survey | 2023 | Pipeline knobs: chunking, top-k, prompting, failure modes. | Survey, not an implementation. |
| [6] | Kurdi et al., AQG review | 2020 | MCQ needs stem, key and distractors; evaluation is hard. | Does not use RAG over a user PDF. |
| [7] | Du et al., neural QG | 2017 | Questions should be conditioned on a passage. | Seq2seq model, not an LLM+index design. |
| [8] | Chen et al., DrQA | 2017 | Retrieve-then-read architecture for document QA. | Sparse retrieval; extractive spans, not free-form answers. |
| [9] | Guu et al., REALM | 2020 | Retriever and language model can be trained together. | Too heavy to retrain in a mini project. |
| [10] | Elkins et al., LLM quizzes | 2024 | Teachers still edit LLM quizzes; Bloom levels are useful. | Not grounded in a private PDF index. |
| [11] | Flesch, readability | 1948 | Simple, explainable reading-ease numbers. | Ignores technical jargon and structure. |
| [12] | Izacard & Grave, FiD | 2021 | Use several passages, not one, when generating. | Encoder–decoder fusion is not our stack. |

*Table 3. Literature mapped to the mini-project gap (twelve papers)*

Two extra observations follow from the table. First, hallucination control in RAG is still an open research topic; our prompt can ask the model to say when the context is insufficient, but we cannot guarantee zero invention. Second, almost none of the papers discuss mobile clients. That part of our work is engineering, not a claim of a new algorithm.

## 7. PROPOSED METHODOLOGY

The methodology is the sequence of steps we will follow to meet the objectives. It is written so that another student group could rebuild a similar system without guessing hidden tricks. Figure-style flow is given as numbered stages; Table 4 lists tools.

## 7.1 Overall approach

We use a three-tier layout. The presentation tier is a React Native application built with Expo. The application tier is a FastAPI service in Python. The data tier is a Pinecone serverless index for vectors and a small SQLite file for session records (readability, glossary and older analyse-text calls). External services are the Gemini API for generation and Hugging Face SentenceTransformers for local embeddings. API keys stay in environment variables, not in source code.

Two paths share the same backend. Path A is the RAG path: ingest PDF, ask questions, generate quizzes from the index. Path B is the direct-text path: a user (or a test client) sends raw text to /analyze; the server returns readability, glossary and heuristic/LLM questions without Pinecone. Path B is useful for unit tests and for short pasted notes. Path A is the main student-facing story.

## 7.2 Stage 1 — PDF intake and text extraction

The client sends a PDF URL to POST /rag/ingest. The server downloads the file with httpx, respecting redirects, and opens it with PyPDF2. Text from each page is concatenated. If the extractor returns an empty string, the API returns an error instead of inserting blank vectors. File upload on POST /analyze/upload is the parallel route for Path B (PDF or plain text). We are aware that multi-column layouts and footers will pollute the text. Cleaning is limited to whitespace normalisation in the first version; a later week can add a header/footer heuristic if demos fail on lecture slides.

## 7.3 Stage 2 — Sentence-aware chunking

Gao et al. [5] treat chunk size as a primary RAG hyperparameter. We split on sentence boundaries (period, question mark, exclamation mark) and pack sentences until the chunk is near 500 words. The last few sentences are carried into the next chunk to provide overlap of about 100 words. Chunks shorter than about 20 words are dropped so that figure captions and page numbers do not become their own vectors. Overlap is a simple way to reduce the chance that a definition and its example land in two unrelated embeddings.

## 7.4 Stage 3 — Embedding and vector storage

Each chunk is prefixed with “passage: ” and encoded with intfloat/e5-large-v2. Vectors are L2-normalised so that cosine similarity in Pinecone is meaningful [4]. Before upsert we check that the dimension is 1024 and that values are finite. Upserts are batched (about 100 vectors) to stay within API limits. Metadata stored with each vector includes the chunk text (truncated if needed), a document id, chunk index, timestamp and source URL. Vector ids follow the pattern {doc_id}-chunk-{index}. The index metric is cosine. If the named index does not exist, the pipeline creates it with dimension 1024.

### 7.5 Stage 4 — Question answering

The client posts a natural-language question to POST /rag/query with top_k (default 5). The question is prefixed with “query: ” and embedded with the same model. Pinecone returns the nearest chunks. The server builds a prompt that contains (a) an instruction to answer only from the supplied context, (b) the concatenated chunk texts with a per-chunk character cap, and (c) the user question. Gemini 2.0 Flash is the preferred generator; Gemini 1.5 Flash is the fallback if the first endpoint fails. Temperature is kept moderate (around 0.7). If the LLM call fails, the API should degrade to returning the retrieved context rather than a silent 500 wherever we can implement that safely. The mobile screen shows the answer and, when available, the supporting snippets so that a student can check the source.

### 7.6 Stage 5 — Quiz generation from the index

POST /rag/quiz accepts num_questions and an optional topic string. The server retrieves a larger set of chunks (on the order of thirty) to obtain coverage, then drops near-duplicate strings. Gemini is asked to return a JSON array of items. Each item is expected to contain a question stem, four options, and the index or letter of the correct option. Parsing must tolerate extra prose around the JSON (the model sometimes wraps output in markdown fences). Invalid items are discarded. A second, simpler quiz path exists in QuizGenerator for Path B: long sentences are turned into questions with a short Gemini prompt, and a heuristic fallback adds a question mark if the API key is missing. Path A is the one we will demonstrate in the PBL viva, because it is document-grounded in the RAG sense [1][6][7].

### 7.7 Stage 6 — Readability and glossary

ReadabilityAnalyzer counts words, sentences and syllables and applies the Flesch Reading Ease and Flesch–Kincaid Grade Level formulae [11]. Syllable counting is a vowel-group heuristic, which is good enough for a prototype. GlossaryBuilder takes frequent tokens of length at least four after removing a small stoplist, then asks Gemini for a one-sentence student

definition and, if requested, a translation. Results of Path B are stored in SQLite through the Database module so that a session can be reloaded.

### 7.8 Stage 7 — Mobile client

The Expo application keeps local state for the PDF URL, the question, ingestion status, the answer, retrieved contexts, and the quiz list. Three main actions map to the three RAG endpoints. The backend URL is read from EXPO_PUBLIC_BACKEND_URL so that an emulator, a tunnel, and a physical phone can point at different hosts without a code change. The UI is intentionally plain: a form, status text, and a scrollable quiz. Visual polish is secondary to a reliable round trip.

### 7.9 Stage 8 — Testing and demo protocol

Testing for a mini project will be scenario-based rather than a large labelled QA benchmark (we do not have Natural Questions annotations for our lecture files). Planned cases: (1) ingest a short public PDF such as a well-known arXiv paper and ask a question whose answer is in the first two pages; (2) ask a question that is unrelated to the PDF and observe whether the model admits lack of support; (3) generate five MCQs and attempt them as a group; (4) upload a short English paragraph to /analyze and check that readability numbers move in the expected direction when sentences are lengthened; (5) stop the Gemini key and confirm that the service still extracts text and, for Path B, still returns heuristic questions. Defects will be logged in the final demo notes.

### 7.10 Tools and hardware

Table 4 lists the planned stack. Hardware is a normal laptop for development and a phone or emulator for the client. Pinecone and Gemini free or student quotas are assumed; if a quota is exhausted we will document the fallback (local context dump without generation).

| Layer | Choice | Role in the project |
|---|---|---|
| Language / API | Python 3.10+, FastAPI, Uvicorn, Pydantic | HTTP API, validation, CORS |
| PDF text | PyPDF2, python-multipart | Extraction and uploads |
| Embeddings | SentenceTransformers, intfloat/e5-large-v2, PyTorch | 1024-d query and passage vectors |
| Vector store | Pinecone (cosine, dim 1024) | ANN search over chunks |
| Generation | Google Gemini 2.0 Flash / 1.5 Flash | Answers and MCQ JSON |
| Local store | SQLite | Analyse sessions |
| Mobile | React Native, Expo SDK 54, TypeScript | Ingest, Q&A, quiz UI |
| Config | python-dotenv, environment variables | API keys and backend URL |

*Table 4. Software stack for the proposed system*

### 7.11 Work split

Sukhada Dhingra will keep the problem statement, literature table and this report consistent with what is actually built, and will define the user flow for the demo. Dipti Kothari will own FastAPI routes, readability and glossary, and SQLite sessions. Khurinji Malar will own the Expo screens, error messages and device testing. Akhil will own chunking, embeddings, Pinecone upsert/query and the RAG quiz prompt, including fallback behaviour. Integration weeks are shared. The split can be adjusted if one module slips, but each name in Table 2 remains responsible for a visible part of the demo.

## 8. TIMELINE OF THE MINI PROJECT

The plan assumes a sixteen-week working window in the odd semester of 2026–27, starting in September 2026. Weeks are calendar weeks, not contact hours. A short review with the guide is expected at the end of Week 4, Week 8 and Week 12. Buffer is kept at the end because API quotas and device networking usually consume extra days.

| Week | Dates (indicative) | Work package | Lead | Deliverable |
|---|---|---|---|---|
| 1 | 08–14 Sep 2026 | Finalise problem, collect 10+ papers, freeze title | Sukhada | This report submitted |
| 2 | 15–21 Sep 2026 | Requirement list, API sketch, Git layout | All | Endpoint list on paper |
| 3 | 22–28 Sep 2026 | FastAPI skeleton, health check, CORS, .env | Dipti | Running /health |
| 4 | 29 Sep–05 Oct 2026 | PDF download and PyPDF2 extraction | Dipti / Akhil | Text dump from sample PDF |
| 5 | 06–12 Oct 2026 | Chunking with overlap; unit checks on length | Akhil | Chunk list for one paper |
| 6 | 13–19 Oct 2026 | E5 embeddings, Pinecone index create/upsert | Akhil | Vectors visible in console |
| 7 | 20–26 Oct 2026 | Query path: embed question, top-k, prompt, Gemini answer | Akhil / Dipti | /rag/query demo |
| 8 | 27 Oct–02 Nov 2026 | Quiz JSON from retrieved chunks; parse/validate | Akhil | /rag/quiz demo |
| 9 | 03–09 Nov 2026 | Readability + glossary + SQLite sessions | Dipti | /analyze on pasted text |
| 10 | 10–16 Nov 2026 | Expo app: ingest form and status | Khurinji | Phone ingest success |
| 11 | 17–23 Nov 2026 | Expo app: Q&A screen and context list | Khurinji | Answer visible on device |
| 12 | 24–30 Nov 2026 | Expo app: quiz | Khurinji | Five-question run |

| | | | | |
|---|---|---|---|---|
| | | attempt UI | | |
| 13 | 01–07 Dec 2026 | Integration, error messages, timeout handling | All | Failure cases documented |
| 14 | 08–14 Dec 2026 | Scenario tests on two PDFs; small prompt fixes | Akhil / Sukhada | Test notes |
| 15 | 15–21 Dec 2026 | Report update, posters/slides, code cleanup | Sukhada / Dipti | Draft final slides |
| 16 | 22–28 Dec 2026 | Buffer, viva rehearsal, submission pack | All | Demo + one group report |

*Table 5. Sixteen-week Gantt-style timeline (indicative dates)*

If the institute's PBL calendar ends earlier than late December, Weeks 13–16 will be compressed into the last two teaching weeks and the buffer will move into study leave. If a public API is blocked, Week 7 will switch to returning retrieved chunks without generation until access is restored. The group will not wait for a perfect model before wiring the mobile client; a mocked JSON response can unblock Khurinji from Week 10.

# 9. EXPECTED OUTCOMES, LIMITATIONS AND CONCLUSION

## 9.1 Expected outcomes

At the end of the mini project we expect: (1) a backend repository with documented environment variables; (2) a mobile client that can ingest one PDF, ask a question and display a quiz; (3) a short live demo on a public PDF; (4) this report, updated if the implementation drifts from the plan. Learning outcomes for the group are practical experience with embeddings, vector search, prompt constraints and cross-platform API use.

## 9.2 Limitations

The first version indexes text only. Scanned books without OCR will fail. Tables and figures are flattened or lost. One index is shared unless we later add metadata filters per document. There is no login, so a classroom deployment would need an extra access layer. Gemini may still write a fluent wrong option even when context is present; retrieval reduces that risk, it does not remove it [1][5]. Readability scores are approximate. Quiz quality will not be measured with a large teacher panel in this semester.

## 9.3 Conclusion

The mini project starts from a narrow, honest problem: PDF notes are hard to query and hard to turn into a fair practice test, while unconstrained language models are willing to invent.

Published work on RAG [1], dense retrieval [2][3][4], surveys of pipeline design [5], educational question generation [6][7][10] and readability [11] gives us a method we can implement with FastAPI, Pinecone, Gemini and React Native in one semester. The objectives in Section 5 are written so that a reviewer can tick them against a demo. The timeline in Section 8 assigns named owners so that the group of four can finish one report and one system, not four disconnected pieces.

## REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. 34th Conf. Neural Information Processing Systems (NeurIPS), 2020.

[2] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019, pp. 3982–3992.

[4] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Text embeddings by weakly-supervised contrastive pre-training,” arXiv:2212.03533, 2022.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.

[6] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” Int. J. Artif. Intell. Educ., vol. 30, no. 1, pp. 121–204, 2020.

[7] X. Du, J. Shao, and C. Cardie, “Learning to ask: Neural question generation for reading comprehension,” in Proc. 55th Annu. Meeting Assoc. Computational Linguistics (ACL), Vancouver, Canada, 2017, pp. 1342–1352.

[8] D. Chen, A. Fisch, J. Weston, and A. Bordes, “Reading Wikipedia to answer open-domain questions,” in Proc. 55th Annu. Meeting Assoc. Computational Linguistics (ACL), Vancouver, Canada, 2017, pp. 1870–1879.

[9] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, “Retrieval augmented language model pre-training,” in Proc. 37th Int. Conf. Machine Learning (ICML), 2020.

[10] S. Elkins, E. Kochmar, J. C. K. Cheung, and I. Serban, “How teachers can use large language models and Bloom’s taxonomy to create educational quizzes,” in Proc. AAAI Conf. Artificial Intelligence, vol. 38, no. 21, pp. 23084–23091, 2024.

[11] R. Flesch, “A new readability yardstick,” J. Appl. Psychol., vol. 32, no. 3, pp. 221–233, 1948.

[12] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in Proc. 16th Conf. European Chapter of the ACL (EACL), 2021, pp. 874–880.

## ANNEXURE A — DECLARATION (TO BE SIGNED BEFORE HARD-COPY SUBMISSION)

We, the members listed in Table 2, declare that this PBL mini-project report is our group's own work. Material taken from papers, books or websites has been cited in the References. We understand that the institute may check the report for plagiarism and that similarity should remain below 10 percent. We also understand that only one report is to be submitted for the group.

Date: _____ Place: Pune

Signatures:

1. Sukhada Dhingra _____

2. Dipti Kothari _____

3. Khurinji Malar _____

4. Akhil _____

## ANNEXURE B — CHECKLIST BEFORE UPLOAD

1. Fill Group Number in Table 1.

2. Fill PRN of every member in Table 2. Do not leave blanks.

3. Fill guide name and semester.

4. Convert this .docx to PDF if the LMS asks for PDF. Keep Times New Roman.

5. Run a similarity check on the PDF. Quoted definitions should stay short; the literature section is already written as commentary, not copied abstracts.

6. Any one member may upload. Inform the other three after upload.